

NORTHERN UNIVERSITY
OF BUSINESS & TECHNOLOGY KHULNA
Knowledge for an Enlightening Future

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Project Proposal

ON

AI Programming Tutor & Bug Explainer

Course Code: CSE 4104

Course Title: Software Development III

SUBMITTED TO

Name: Md. Riaz Mahmud

Assistant Professor

Department of CSE

Northern University of Business & Technology, Khulna.

SUBMITTED BY

Team Name	CSE4104 - 7B - T04
Section	7B
Project Title	AI Programming Tutor & Bug Explainer
Member Names	M.M. Asfy Or Rayhan Alisha Rahman Nadia Afrin Fatima Tanjim Rafa
Student IDs	11230121106 11230121112 11230121121 11230321556
GitHub Repository	https://github.com/Rayhan-55/csc4104-7b-t04-ai-programming-tutor-and-Bug-Explainer

Date of Submission: 25/05/2026

Contents

1. **Problem Statement**.....1

2. **Objectives**.....3

3. **Features**.....5

4. **Technology Stack**.....12

5. **Expected Outcome**.....16

Figures

1. **Figure 1: System flow of AI Programming Tutor & Bug Explainer**.....11

Project Title: AI Programming Tutor & Bug Explainer

1. Problem Statement

1.1 The Existing Learning Challenges

Learning programming is one of the most cognitively demanding skills a student can undertake. For learners in Bangladesh and other developing regions, this challenge is amplified significantly by language barriers, limited access to quality tutors, and the highly technical nature of compiler errors and debugging messages. When beginner students encounter errors such as "Segmentation fault (core dumped)" or "NullPointerException," they often fail to understand what the error actually means or how to solve it. Most online explanations are written for experienced developers and use advanced technical terminology. Additionally, many educational materials are written entirely in academic English, which becomes difficult for students who are more comfortable with Bangla or Banglish communication styles.

1.2 Importance of the Problem

Programming literacy has become an essential skill in the modern digital era. However, a large number of students struggle during introductory programming courses due to the lack of personalized and beginner-friendly guidance. In Bangladesh, the situation is more challenging because skilled programming mentors are limited and mostly concentrated in urban areas. Many students cannot afford private tutoring or do not have access to interactive learning support outside the classroom. As a result, students lose confidence, become frustrated with debugging problems, and eventually lose interest in programming.

1.3 Limitations of Existing Solutions

Existing learning platforms and tools still fail to fully support beginner programmers effectively.

- Generic AI chatbots such as OpenAI's ChatGPT or Google Bard provide technically correct answers but are often too advanced and overwhelming for beginners.

- Online learning platforms such as Coursera and Udemy mainly offer pre-recorded courses and cannot provide instant debugging assistance for a student's own code.
- Platforms like Stack Overflow require prior technical knowledge to search and understand solutions effectively.
- Bangla-language programming resources are limited and rarely focus on beginner-level debugging support.
- Most existing systems do not maintain learner history or adapt responses based on the student's weaknesses and progress.

1.4 Solution Provided by AI Programming Tutor & Bug Explainer

AI Programming Tutor & Bug Explainer addresses these challenges by providing a dedicated bilingual AI-based learning assistant specially designed for beginner programmers.

The system offers:

- Beginner-friendly explanations using a structured teaching format
- Translation of compiler and runtime errors into simple Bangla and Banglish
- Personalized learning support through persistent learner memory
- AI-powered analysis of PDFs, screenshots, and handwritten notes
- Interactive debugging assistance for real coding problems
- Low-cost accessibility using free AI providers and offline demo support.

2. Objectives

The main objective of the AI Programming Tutor & Bug Explainer project is to develop an intelligent, beginner-friendly, and interactive programming learning platform that helps students overcome common difficulties in coding, debugging, and understanding programming concepts. The system aims to create a personalized learning environment where students can receive instant AI-powered assistance in both Bangla and English languages. The project is designed to reduce the fear and frustration beginners face while learning programming by simplifying complex technical concepts, compiler errors, and debugging processes into understandable explanations. Another important objective is to make programming education more accessible for students who do not have continuous access to expert mentors or premium educational resources. In addition, the project focuses on integrating modern AI technologies, learner progress tracking, and interactive educational tools into a single platform that supports practical learning, self-improvement, and long-term skill development.

The primary objectives of the AI Programming Tutor & Bug Explainer project are:

- To design and implement a full-stack AI-powered programming tutor web application for beginner-level computer science students.
- To develop a bilingual (Bangla and English) AI interaction system that adapts responses according to the learner's language preference.
- To build a Compiler Error Translator that converts technical compiler and runtime errors into simple beginner-friendly explanations with solution guidance.
- To create a Bug Explanation System that analyzes student code, identifies logical or syntax errors, and provides corrected code with detailed explanations.
- To implement a persistent AI Memory System that tracks user progress, weak topics, and previous learning history for personalized tutoring.
- To integrate a GitHub Repository Explainer that helps students understand open-source projects through repository URL analysis.

- To provide a Document and Image Explainer capable of analyzing PDFs, screenshots, and handwritten notes using AI.
- To implement gamification features such as XP points, streak systems, activity tracking, and level progression to increase learning motivation.
- To build a secure role-based admin panel for monitoring users, managing content, and analyzing platform activity.
- To develop a provider-independent AI architecture that supports multiple AI services such as OpenAI, Google Gemini, Groq, and offline demo mode without major system modification.
- To encourage self-learning and independent problem-solving skills among beginner programmers.
- To create an affordable and accessible AI learning solution for students in Bangladesh and other developing regions.
- To improve students' confidence and programming problem-solving ability through continuous AI-guided support.

3. Features

3.1 Authentication & Role-Based Access Control

The AI Programming Tutor & Bug Explainer implements a complete and secure JWT-based authentication system to ensure safe and role-specific access across the platform. Users register using basic credentials such as name, email, and password, where passwords are securely hashed using bcryptjs with a cost factor of 10 before being stored in the database. Upon successful login, the system generates a signed JSON Web Token (JWT), which is stored on the client side and attached to every protected request. A dedicated authentication middleware verifies the token before granting access to secured routes, ensuring that only valid users can interact with protected resources. The system supports two user roles: student (default role) and admin. Admin access is managed through environment-based configuration (ADMIN_EMAILS) for automatic promotion during first login, while additional role control is available through the admin panel where administrators can promote or demote users dynamically. On the frontend, protected routes are handled using a Protected Route component that redirects unauthenticated users to the login page, ensuring complete route-level security.

3.2 AI Coding Tutor Chatbot

The AI Coding Tutor Chatbot is the core learning interface of the AI Programming Tutor & Bug Explainer platform. It allows students to ask programming-related questions in natural language using Bangla, English, or Banglish without needing to know formal technical syntax.

The AI response is strictly structured into four educational parts to ensure clarity and consistency:

1. A simple one-line definition of the concept
2. A real-life analogy to make the concept intuitive
3. A minimal working code example to demonstrate usage
4. A short motivational or encouraging sentence to support learning

This structured output is enforced through prompt engineering in the system prompts (prompts.js), ensuring that every response follows a beginner-friendly teaching pattern. Additionally, when user memory exists, the system injects contextual information such as previously weak topics or past queries, enabling adaptive learning behavior. For example, the tutor may reference earlier mistakes and connect them to the current topic to reinforce understanding.

3.3 Bug Explanation System

The Bug Explanation System is designed to transform debugging from a frustrating experience into a structured learning process. Students paste their source code into a Monaco Editor-based interface along with the programming language (such as C, C++, Java, Python, or JavaScript).

The AI then analyzes the code and returns a detailed breakdown consisting of:

1. A simple explanation of what the bug is and why it occurs
2. A line-by-line analysis highlighting exactly where the issue exists
3. A fully corrected version of the code inside a properly formatted code block
4. A set of prevention tips to help students avoid similar mistakes in the future

Unlike traditional debugging tools that only return error fixes, this system prioritizes conceptual understanding, ensuring that students learn the underlying cause of the issue rather than simply copying corrected code.

3.4 Compiler & Runtime Error Translator (Flagship Feature)

This is the most advanced and unique feature of the AI Programming Tutor & Bug Explainer system. It focuses on one of the biggest challenges faced by beginners: understanding cryptic compiler and runtime errors. Students paste raw error messages (such as segmentation faults, stack overflow errors, null pointer exceptions, or JavaScript type errors) along with the programming language used. The AI then converts these technical messages into a structured, beginner-friendly explanation consisting of:

1. A simple interpretation of what the error means in plain language
2. The most common causes behind the error
3. Step-by-step instructions to fix the issue
4. A small illustrative example when necessary

This feature significantly reduces confusion by translating machine-level error messages into human-understandable explanations, particularly useful for first-year programming students.

3.5 AI Code Improvement Suggestions

The Code Improvement module is designed to refine existing working code without over-complicating or unnecessarily rewriting it. The goal is to guide beginners toward better coding practices gradually.

The system analyzes submitted code and suggests improvements such as:

- More meaningful and descriptive variable names
- Cleaner and more readable code structure
- Better algorithmic efficiency (e.g., replacing nested loops with hash-based structures where appropriate)
- Removal of redundant or unnecessary logic
- Language-specific best practice recommendations

Each suggestion is accompanied by a clear explanation describing why the improvement matters and how it affects readability, performance, or maintainability. The explanations are often simplified using Banglish to ensure accessibility for all learners.

3.6 GitHub Repository Explainer

The GitHub Repository Explainer allows students to understand any open-source project simply

by providing its GitHub URL. The system uses the GitHub REST API to fetch repository metadata, README content, and file structure information.

Once the data is collected, the AI generates a structured explanation including:

1. A short and simple overview of the project
2. The purpose of the project and its target users
3. Technologies identified from the repository structure
4. Key files and their functional descriptions
5. A suggested method to run or set up the project locally

Additionally, students can explore individual files and receive AI-generated explanations for each file, making it easier to understand large-scale open-source projects step by step.

3.7 PDF & Image Document Explainer

This feature enables students to interact directly with academic materials in a conversational way. Users can upload PDFs, lecture slides, textbook pages, screenshots of code, or even handwritten notes. For PDF files, the system extracts text using the unpdf library without requiring external dependencies. For image-based inputs, multimodal AI models (such as Gemini Vision or equivalent APIs) are used by converting images into base64 format for analysis. Students can also ask direct questions about the uploaded content, and the AI prioritizes these queries when generating responses. This makes static learning materials fully interactive and searchable in real time. Additionally, the system can summarize long lecture notes into concise key points for quick revision. It supports multi-page document understanding, allowing students to ask questions across the entire file context. The AI can also explain complex diagrams or handwritten code step-by-step in simple Bangla or English. This significantly improves accessibility for students who struggle with dense academic materials.

3.8 AI Memory & Personalization System

The AI Memory System is one of the most important components that differentiates AI Programming Tutor & Bug Explainer from generic AI tools. It enables persistent, personalized learning across sessions. Every user interaction is stored as structured memory notes, with a controlled limit to maintain performance. The system also tracks "weak topics" using a separate model that records mistake frequency and confidence level for each concept. When generating responses, the AI retrieves recent memory entries and weak topic data, allowing it to adapt explanations based on the student's learning history. This creates a tutoring experience that evolves over time, similar to a human mentor who remembers past struggles and adjusts teaching style accordingly.

3.9 Student Dashboard with Gamification

The student dashboard provides a complete overview of learning progress and engagement metrics. It displays XP points earned through platform usage, user level progression based on XP thresholds, and daily learning streaks that encourage consistency. An interactive activity chart visualizes user engagement over time using Recharts, showing how frequently different features are used. The dashboard also includes a history section that logs recent AI interactions, including queries, feature usage, and response previews. Gamification elements such as XP rewards, streak tracking, and level progression are integrated to increase motivation and encourage continuous daily learning.

3.10 Administrative Panel

The administrative panel provides full control and visibility over platform activity for users with admin privileges. It includes system-wide analytics such as total user count, total AI interactions, and overall platform usage statistics. Admins can view feature-wise usage distribution through graphical charts, identify top-performing users through XP-based leaderboards, and manage user roles by promoting or demoting students. The panel also includes a live activity feed showing recent interactions across the entire system. This ensures that instructors or system administrators can monitor engagement, identify usage trends, and maintain platform integrity effectively.

3.11 Voice Tutor

The Voice Tutor feature enhances accessibility by converting AI-generated text responses into speech using the Web Speech API. This allows students to listen to explanations instead of reading them, which is particularly helpful for auditory learners. Since it is implemented entirely using browser-native functionality, it requires no additional API keys or backend services, making it lightweight, fast, and universally available to all users.

3.12 Practice Problem Generator

The Practice Problem Generator provides structured coding exercises for students to improve their skills through active learning. The backend AI is configured to return responses in strict JSON format, ensuring consistency and frontend compatibility. Each generated problem includes a title, problem statement, sample inputs and outputs, hints, a fully commented solution, and a simplified explanation of the solution approach. This structured format allows students to learn step-by-step problem-solving techniques in a guided manner.

3.13 Provider-Agnostic AI Abstraction Layer

The system is built with a flexible AI architecture that decouples application logic from specific AI providers. A central service (aiService.js) handles all AI requests through a unified function, allowing seamless switching between providers such as OpenAI, Google Gemini, Groq, or a local mock mode. This abstraction ensures that the system remains fully functional even if external API services are unavailable. In such cases, the system automatically falls back to a mock AI response generator that produces realistic Banglish-style outputs, allowing uninterrupted demonstration of all features without external dependencies.

AI Programming Tutor & Bug Explainer

System Flow

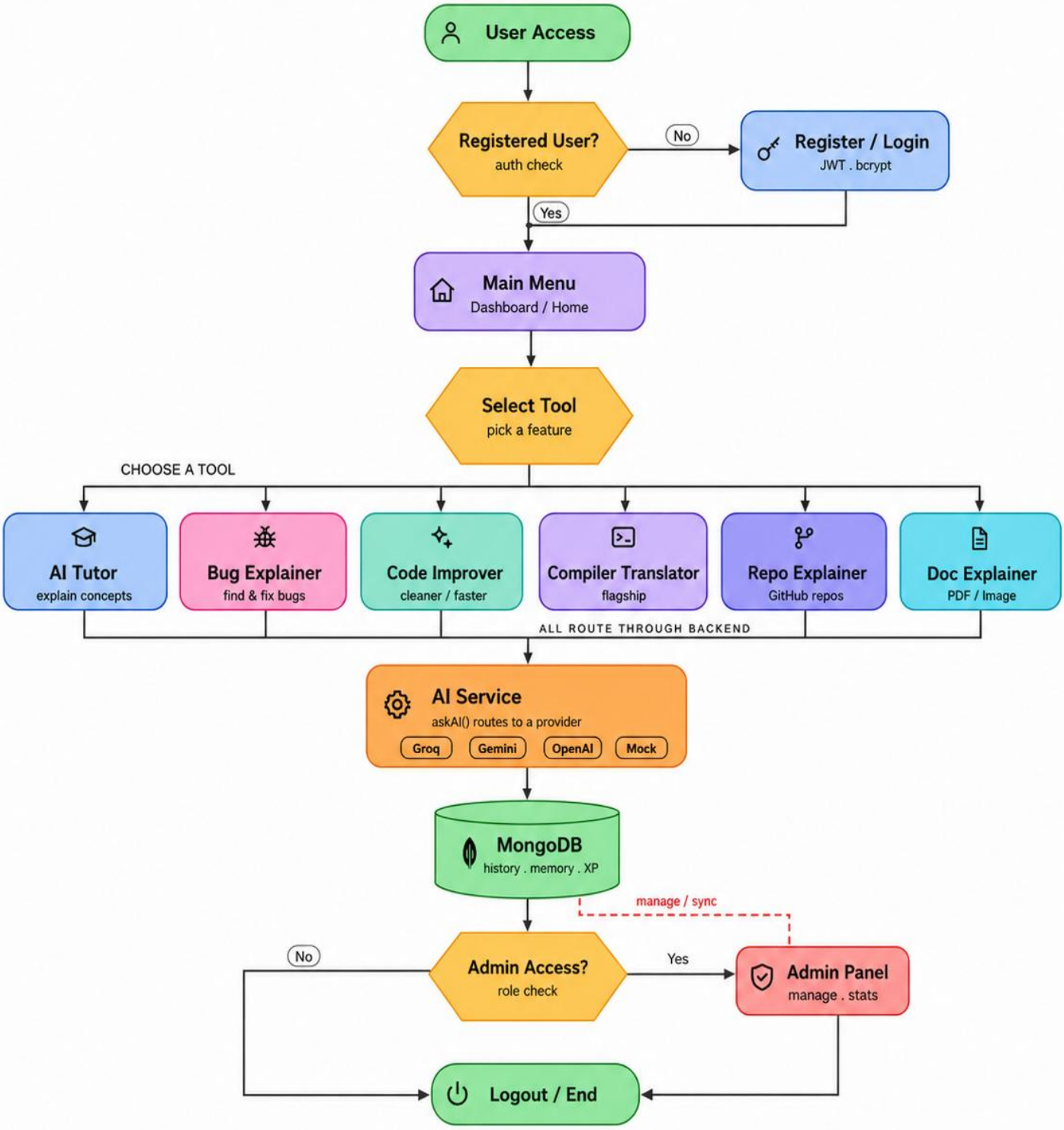


Figure 1: System flow of AI Programming Tutor & Bug Explainer

4. Technology Stack

The AI Programming Tutor & Bug Explainer is built using a modern full-stack architecture designed for scalability, performance, and maintainability. The system separates frontend, backend, database, and AI services into independent layers, allowing easy updates and flexible provider switching. The overall stack is optimized for fast development, smooth user experience, and integration of AI-driven features such as chat tutoring, debugging, and document analysis.

4.1 Frontend

Technology	Purpose / Notes
React 18	Component-based UI library for building the SPA
Vite 5	Build tool and dev server; fast HMR in development
React Router DOM v6	Client-side routing with ProtectedRoute wrappers
Tailwind CSS 3	Utility-first CSS framework for responsive styling
Axios	HTTP client for backend API communication
@monaco-editor/react	VS Code-grade code editor embedded in Bug Explainer and Code Improver pages
Recharts	Chart library for dashboard activity chart and admin feature-usage bar chart
Framer Motion	Animation library for page transitions and UI microinteractions

Web Speech API	Browser-native TTS for the Voice Tutor feature (no library required)

4.2 Backend

Technology	Purpose / Notes
Node.js 18+	JavaScript runtime for the server (ES Module syntax)
Express.js 4	HTTP web framework; REST API routing
Mongoose 8	ODM for MongoDB; schema definitions and query API
bcryptjs	Password hashing (salt rounds = 10)
jsonwebtoken	JWT generation and verification for auth
multer 2	Multipart file upload handler for PDF/image uploads
unpdf	Server-side PDF text extraction without binary dependencies
express-rate-limit	Request rate limiting on AI endpoints to prevent abuse
cors	Cross-origin resource sharing configuration
dotenv	Environment variable management
nodemon	Development auto-restart on file changes

4.3 Database

Technology	Purpose / Notes
MongoDB (Atlas)	Primary NoSQL database; free tier sufficient for the application
Mongoose Schemas	User, ChatHistory, WeakTopic, Memory models with indexes

4.4 AI Providers & APIs

Provider / API	Purpose / Notes
Groq API (Llama 3.3 70B)	Primary free-tier text AI provider; OpenAI-compatible API format
Google Gemini 1.5 Flash	Alternative text + vision AI provider; free tier available
OpenAI API (GPT-4o-mini)	Optional premium text + vision provider
GitHub REST API v3	Fetches repo metadata, README, and file tree for Repo Explainer; no token required (60 req/hr)
Mock Provider	Built-in offline fallback with realistic Banglish placeholder responses

4.5 Deployment Targets

Platform	Purpose / Notes
Vercel	Frontend deployment (Vite SPA)
Render / Railway	Backend deployment (Node.js Express server)
MongoDB Atlas	Cloud-hosted database (free M0 cluster)

Overall, the AI Programming Tutor & Bug Explainer uses a modern full-stack technology architecture that combines React-based frontend, Node.js backend, and MongoDB database to ensure a fast, secure, and scalable system. It integrates multiple AI providers through a unified layer for reliable intelligent responses, along with tools like GitHub API and document processing for advanced learning features. The system is deployed using cloud platforms like Vercel, Render, and MongoDB Atlas, making it fully production-ready and easily accessible.

5. Expected Outcome

5.1 Direct User Benefits

Students using the AI Programming Tutor & Bug Explainer are expected to experience a significant improvement in how they learn programming and solve coding problems. The platform provides 24/7 access to an AI tutor that responds in a patient and supportive manner, helping remove the fear and hesitation that often prevents beginners from asking basic questions. One of the most impactful benefits is the ability to understand compiler and runtime errors in plain Bangla or Banglish within seconds, instead of spending a long time searching through complex forums or documentation. Over time, the system becomes more personalized by learning from user interactions and weak areas, which allows it to deliver increasingly tailored explanations and guidance. In addition, since the entire platform is built on free-tier technologies and APIs, it remains financially accessible to students from all backgrounds, while also allowing flexible communication in the user's preferred language style.

5.2 Improvement to Educational Workflow

For educators and institutions, the AI Programming Tutor & Bug Explainer provides meaningful support in improving the teaching and learning process. The administrative dashboard offers insights into student activity, frequently used features, and common weak topics, which can help instructors identify difficult areas in the curriculum and adjust teaching strategies accordingly. The system also reduces repetitive beginner-level queries by handling common programming doubts automatically, allowing teachers to focus more on advanced concepts and interactive classroom engagement. Furthermore, it extends learning beyond classroom hours by providing continuous AI-driven assistance, ensuring that students can practice and get help even outside scheduled academic time.

5.3 Technical & Academic Contributions

From a software engineering perspective, the AI Programming Tutor & Bug Explainer demonstrates several important architectural and academic contributions. It implements a provider-agnostic AI abstraction layer that decouples application logic from specific AI vendors,

effectively applying the Adapter design pattern in a real-world LLM-based system. The project also treats prompt engineering as a structured and maintainable component of software development, centralizing all AI instructions to ensure consistency and scalability. Another key contribution is the memory-augmented learning approach, where user history and weak-topic tracking are used to simulate lightweight personalized retrieval without requiring complex vector databases. Additionally, the system integrates multimodal input handling by combining text extraction from PDFs and vision-based analysis for images, enabling a unified learning experience across different types of educational content.

5.4 Future Impact & Roadmap

The AI Programming Tutor & Bug Explainer is designed with a highly extensible architecture that allows future enhancements without major structural changes. Planned improvements include an interactive practice arena built on the existing problem generation API, automatic weak-topic detection through deeper analysis of chat history, and an AI-powered mock interview system to simulate real technical interview scenarios. Additional scalability features such as Redis caching for frequently asked questions and a daily challenge leaderboard can further enhance performance and engagement. There is also potential for integrating a secure code execution sandbox to allow real-time code testing and output visualization. Overall, the system provides a strong and scalable foundation for a next-generation AI-powered educational platform tailored for programming learners.